C++ STL

Guia de Referência para Programação Competitiva

Maratona de Programação

Sumário

1 Convenções **3**
- 1.1 Notações utilizadas . . . 3
- 1.2 Intervalos . . . 3
- 1.3 Complexidade . . . 3

2 Introdução à STL **4**
- 2.1 Containers . . . 4
- 2.2 Iteradores . . . 4
- 2.3 Algoritmos . . . 4
- 2.4 Cabeçalhos . . . 5
- 2.5 Primeiro exemplo . . . 5

3 Containers **6**
- 3.1 Operações comuns . . . 6
- 3.2 Containers sequenciais . . . 6
  - 3.2.1 `vector` . . . 6
  - 3.2.2 `array` . . . 7
  - 3.2.3 `deque` . . . 8
  - 3.2.4 `list` . . . 9
- 3.3 Containers associativos ordenados . . . 9
  - 3.3.1 `set` . . . 9
  - 3.3.2 `multiset` . . . 10
  - 3.3.3 `map` . . . 10
  - 3.3.4 `multimap` . . . 11
- 3.4 Containers associativos não ordenados . . . 11
  - 3.4.1 `unordered_set` . . . 12
  - 3.4.2 `unordered_multiset` . . . 12
  - 3.4.3 `unordered_map` . . . 12
  - 3.4.4 `unordered_multimap` . . . 13

4 Iteradores **13**
- 4.1 `begin()` e `end()` . . . 13
- 4.2 Iteradores reversos . . . 13
- 4.3 Categorias de iteradores . . . 13
- 4.4 Iteradores retornados por operações . . . 14

5 Adaptadores de containers **14**
5.1 `stack` . . . 14
5.2 `queue` . . . 15
5.3 `priority_queue` . . . 15

6 Estruturas auxiliares **16**
6.1 `pair` . . . 16
6.2 `tuple` . . . 17

7 Objetos-função e comparadores **18**
7.1 `less` . . . 18
7.2 `greater` . . . 18
7.3 Comparadores personalizados . . . 19

8 Algoritmos **19**
8.1 Algoritmos de busca e consulta . . . 19
8.1.1 `find` . . . 19
8.1.2 `count` . . . 20
8.1.3 `min_element` e `max_element` . . . 20
8.2 Ordenação e busca binária . . . 20
8.2.1 `sort` . . . 20
8.2.2 `binary_search` . . . 20
8.2.3 `lower_bound` e `upper_bound` . . . 20
8.3 Manipulação de sequências . . . 21
8.3.1 `reverse` . . . 21
8.3.2 `rotate` . . . 21
8.3.3 `unique` . . . 22
8.4 Permutações . . . 22
8.4.1 `next_permutation` . . . 22
8.4.2 `prev_permutation` . . . 23
8.5 Operações numéricas . . . 23
8.5.1 `accumulate` . . . 24
8.5.2 `iota` . . . 24
8.6 Operações sobre conjuntos . . . 24
8.6.1 `set_union` . . . 24
8.6.2 `set_intersection` . . . 24
8.6.3 `set_difference` . . . 25

9 Escolhendo estruturas e algoritmos **26**

10 Problemas de programação competitiva **26**
10.1 Ordenando as pontuações . . . 26
10.2 Contando valores distintos . . . 27
10.3 Frequência dos valores . . . 28
10.4 Atendimento por prioridade . . . 29
10.5 Primeira posição possível . . . 30
10.6 Busca em largura em um grafo . . . 30
10.7 Menor caminho com fila de prioridade . . . 31
10.8 Soma de prefixos . . . 32

11 Referência rápida **33**

12 Considerações finais **35**

1 Convenções

Este guia utiliza algumas convenções para simplificar a apresentação das estruturas, operações e algoritmos da STL.

1.1 Notações utilizadas

Notação	Significado
`C`	um container da STL
`v`	um `vector`
`it`	um iterador
`first`	iterador para o início de um intervalo
`last`	iterador para o final de um intervalo
`x`	um elemento
`n`	quantidade de elementos
`T`	tipo dos elementos armazenados

1.2 Intervalos

Muitos algoritmos da STL trabalham sobre um intervalo determinado por dois iteradores:

$$[\texttt{first}, \texttt{last})$$

O elemento indicado por `first` pertence ao intervalo, enquanto o elemento indicado por `last` não pertence.

Assim, no comando:

```
1 sort(v.begin(), v.end());
```

`v.begin()` indica o primeiro elemento do vetor e `v.end()` indica a posição imediatamente posterior ao último elemento.

1.3 Complexidade

A complexidade indica como o tempo de execução de uma operação cresce em função da quantidade de elementos.

Complexidade	Interpretação
$O(1)$	tempo constante
$O(\log n)$	tempo logarítmico
$O(n)$	tempo linear
$O(n \log n)$	comum em algoritmos eficientes de ordenação
$O(n^2)$	tempo quadrático

Em programação competitiva, a complexidade das operações deve ser considerada na escolha das estruturas utilizadas na solução.

2 Introdução à STL

A *Standard Template Library* (STL) reúne estruturas de dados e algoritmos disponíveis na biblioteca padrão da linguagem C++.

A STL oferece implementações eficientes e padronizadas de diversas estruturas frequentemente utilizadas em programação, permitindo que o programador concentre seus esforços na elaboração do algoritmo que resolve o problema.

Em programação competitiva, a STL é particularmente importante por reduzir a quantidade de código necessária para implementar soluções e por disponibilizar estruturas e algoritmos amplamente testados.

Entre seus principais componentes destacam-se:

- containers;
- iteradores;
- adaptadores de containers;
- objetos-função;
- algoritmos.

2.1 Containers

Containers são estruturas responsáveis pelo armazenamento e pela organização de conjuntos de elementos.

Entre os containers mais utilizados estão:

```
vector<int>
deque<int>
set<int>
map<int, int>
unordered_map<int, int>
```

2.2 Iteradores

Iteradores representam posições dentro dos containers e permitem percorrer seus elementos.

Por exemplo:

```
vector<int>::iterator it;
```

ou, utilizando inferência de tipo:

```
auto it = v.begin();
```

2.3 Algoritmos

A STL fornece algoritmos capazes de operar sobre diferentes containers por meio de iteradores.

Alguns exemplos são:

```
sort(v.begin(), v.end());
reverse(v.begin(), v.end());
find(v.begin(), v.end(), x);
```

2.4 Cabeçalhos

Cada componente da biblioteca está definido em um cabeçalho específico.

```
#include <vector>
#include <queue>
#include <set>
#include <map>
#include <algorithm>
```

Em programação competitiva também é comum utilizar:

```
#include <bits/stdc++.h>
```

Esse cabeçalho é disponibilizado pela implementação da biblioteca padrão C++ utilizada pelo GCC e inclui grande parte de seus cabeçalhos. Entretanto, ele não faz parte oficialmente do padrão C++.

2.5 Primeiro exemplo

O programa a seguir utiliza um container, iteradores e um algoritmo da STL.

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
    vector<int> v = {8, 3, 5, 1, 9};

    sort(v.begin(), v.end());

    for (int x : v)
        cout << x << " ";

    cout << endl;

    return 0;
}
```

Nesse exemplo:

- `vector` é o container responsável por armazenar os valores;
- `begin()` e `end()` fornecem os iteradores que delimitam o intervalo;
- `sort` é o algoritmo aplicado ao intervalo.

A saída produzida é:

```
1 3 5 8 9
```

Esse modelo de interação entre containers, iteradores e algoritmos constitui uma das principais características da STL.

3 Containers

Containers são estruturas utilizadas para armazenar conjuntos de elementos. Diferentes containers oferecem diferentes formas de organização e diferentes complexidades para operações como acesso, busca, inserção e remoção.

Neste guia, os containers serão divididos em:

- containers sequenciais;
- containers associativos ordenados;
- containers associativos não ordenados.

3.1 Operações comuns

Diversos containers possuem operações semelhantes.

Operação	Descrição
`C.size()`	quantidade de elementos
`C.empty()`	verifica se está vazio
`C.clear()`	remove todos os elementos
`C.begin()`	iterador para o primeiro elemento
`C.end()`	iterador após o último elemento

3.2 Containers sequenciais

Containers sequenciais armazenam seus elementos segundo uma sequência. Entre os principais estão `vector`, `array`, `deque` e `list`.

3.2.1 `vector`

O `vector` representa uma sequência cujo tamanho pode ser alterado dinamicamente. Seus elementos são armazenados de forma contígua na memória, permitindo acesso eficiente por índice.

Biblioteca

```
1 #include <vector>
```

Declaração

```
1 vector<int> v;                 // Vetor vazio
2 vector<int> v(10);             // 10 elementos inicializados com 0
3 vector<int> v(10, 5);          // 10 elementos com valor 5
4 vector<int> v = {10, 20, 30};  // Vetor inicializado com esses valores
```

Principais operações

Operação	Descrição	Complexidade
`v[i]`	acessa a posição i	$O(1)$
`v.at(i)`	acessa a posição i com verificação	$O(1)$
`v.front()`	primeiro elemento	$O(1)$
`v.back()`	último elemento	$O(1)$
`v.push_back(x)`	insere no final	$O(1)$ amortizado
`v.pop_back()`	remove do final	$O(1)$
`v.size()`	quantidade de elementos	$O(1)$
`v.empty()`	verifica se está vazio	$O(1)$
`v.clear()`	remove todos os elementos	$O(n)$

Exemplo

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> v;

    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    for (int x : v)
        cout << x << " ";

    cout << endl;

    return 0;
}
```

Uso em programação competitiva

O `vector` é normalmente a primeira opção para armazenar uma sequência quando são necessários acesso por índice, percorrimento eficiente e alteração dinâmica do tamanho.

3.2.2 `array`

O `array` representa uma sequência de tamanho fixo.

Biblioteca

```cpp
#include <array>
```

Declaração

```cpp
array<int, 5> v;
array<int, 5> v = {10, 20, 30, 40, 50};
```

Exemplo

```cpp
#include <iostream>
#include <array>

using namespace std;

int main() {
    array<int, 5> v = {10, 20, 30, 40, 50};

    for (int x : v)
        cout << x << " ";

    cout << endl;

    return 0;
}
```

Seu tamanho é conhecido em tempo de compilação e não pode ser alterado durante a execução.

Quando usar `vector` ou `array`?

Tanto `vector` quanto `array` armazenam seus elementos de forma contígua e permitem acesso direto por índice em $O(1)$. A principal diferença está no tamanho da estrutura: em um `array`, o tamanho é fixo e definido na declaração, enquanto um `vector` pode aumentar ou diminuir durante a execução do programa. Assim, quando o tamanho da sequência é conhecido e não será alterado, pode-se utilizar `array`; quando o tamanho precisa ser definido ou modificado durante a execução, `vector` é normalmente a escolha mais adequada.

3.2.3 `deque`

O `deque`, abreviação de *double-ended queue*, permite inserções e remoções eficientes nas duas extremidades.

Biblioteca

```cpp
#include <deque>
```

Principais operações

Operação	Descrição	Complexidade
`push_front(x)`	insere no início	$O(1)$
`push_back(x)`	insere no final	$O(1)$
`pop_front()`	remove do início	$O(1)$
`pop_back()`	remove do final	$O(1)$
`front()`	primeiro elemento	$O(1)$
`back()`	último elemento	$O(1)$
`operator[]`	acesso por índice	$O(1)$

Exemplo

```cpp
#include <iostream>
#include <deque>

using namespace std;
```

```
5
6  int main() {
7      deque<int> d;
8
9      d.push_back(20);
10     d.push_front(10);
11     d.push_back(30);
12
13     for (int x : d)
14         cout << x << " ";
15
16     cout << endl;
17
18     return 0;
19 }
```

3.2.4 `list`

A `list` representa uma lista duplamente encadeada.

Biblioteca

```
1  #include <list>
```

Principais operações

```
1  list<int> l;
2
3  l.push_front(10);
4  l.push_back(20);
5
6  l.pop_front();
7  l.pop_back();
```

A estrutura permite inserções e remoções em tempo constante quando a posição já é conhecida por meio de um iterador. Entretanto, não oferece acesso direto por índice.

Em programação competitiva, seu uso é menos frequente que o de `vector` e `deque`.

3.3 Containers associativos ordenados

Containers associativos armazenam seus elementos de acordo com uma chave. Nos containers associativos ordenados, as chaves são mantidas em ordem.

3.3.1 `set`

O `set` representa um conjunto ordenado de elementos distintos.

Biblioteca

```
1  #include <set>
```

Principais operações

Operação	Descrição	Complexidade
`insert(x)`	insere um elemento	$O(\log n)$
`erase(x)`	remove um elemento	$O(\log n)$
`find(x)`	procura um elemento	$O(\log n)$
`count(x)`	verifica a ocorrência	$O(\log n)$

Exemplo

```
#include <iostream>
#include <set>

using namespace std;

int main() {
    set<int> s;

    s.insert(30);
    s.insert(10);
    s.insert(20);
    s.insert(10);

    for (int x : s)
        cout << x << " ";

    cout << endl;

    return 0;
}
```

A saída é:

```
10 20 30
```

O valor 10 não é armazenado novamente, pois o `set` não permite elementos repetidos.

3.3.2 `multiset`

O `multiset` possui características semelhantes ao `set`, mas permite elementos repetidos.

```
multiset<int> s;

s.insert(10);
s.insert(10);
s.insert(20);
```

As operações de inserção, busca e remoção possuem complexidade $O(\log n)$.

3.3.3 `map`

O `map` armazena pares formados por uma chave e um valor. Cada chave é única e as chaves são mantidas ordenadas.

Biblioteca

```
#include <map>
```

Declaração

```
map<string, int> idade;
map<int, int> frequencia;
```

Principais operações

Operação	Descrição	Complexidade
`m[chave]`	acessa ou cria uma chave	$O(\log n)$
`find(chave)`	procura uma chave	$O(\log n)$
`count(chave)`	verifica uma chave	$O(\log n)$
`erase(chave)`	remove uma chave	$O(\log n)$

Exemplo

```cpp
#include <iostream>
#include <map>
#include <string>

using namespace std;

int main() {
    map<string, int> idade;

    idade["Ana"] = 20;
    idade["Carlos"] = 22;
    idade["Maria"] = 19;

    for (auto p : idade)
        cout << p.first << " "
             << p.second << endl;

    return 0;
}
```

Um uso frequente em programação competitiva é a contagem de frequências:

```cpp
map<int, int> freq;

for (int x : v)
    freq[x]++;
```

3.3.4 `multimap`

O `multimap` permite que uma mesma chave esteja associada a diferentes valores.

```cpp
multimap<int, string> m;

m.insert({1, "Ana"});
m.insert({1, "Carlos"});
m.insert({2, "Maria"});
```

3.4 Containers associativos não ordenados

Os containers associativos não ordenados utilizam tabelas de dispersão (*hash tables*). Seus elementos não são mantidos em ordem.

3.4.1 `unordered_set`

O `unordered_set` armazena elementos distintos sem manter uma ordem específica.

Biblioteca

```
#include <unordered_set>
```

Exemplo

```
#include <iostream>
#include <unordered_set>

using namespace std;

int main() {
    unordered_set<int> s;

    s.insert(10);
    s.insert(20);
    s.insert(30);

    if (s.find(20) != s.end())
        cout << "Encontrado" << endl;

    return 0;
}
```

Inserção, remoção e busca possuem complexidade esperada $O(1)$, embora no pior caso possam atingir $O(n)$.

3.4.2 `unordered_multiset`

O `unordered_multiset` possui as mesmas características gerais do `unordered_set`, mas permite elementos repetidos.

3.4.3 `unordered_map`

O `unordered_map` associa chaves a valores utilizando uma tabela de dispersão.

Biblioteca

```
#include <unordered_map>
```

Exemplo

```
#include <iostream>
#include <unordered_map>

using namespace std;

int main() {
    unordered_map<int, int> freq;

    freq[10]++;
    freq[20]++;
    freq[10]++;

    cout << freq[10] << endl;
```

```
14
15      return 0;
16  }
```

Busca, inserção e remoção possuem complexidade esperada $O(1)$.

3.4.4 `unordered_multimap`

O `unordered_multimap` permite múltiplos valores associados à mesma chave e não mantém suas chaves ordenadas.

4 Iteradores

Iteradores são objetos utilizados para indicar posições dentro de containers. Eles permitem que algoritmos sejam escritos de forma independente da estrutura utilizada para armazenar os elementos.

4.1 `begin()` e `end()`

A função `begin()` retorna um iterador para o primeiro elemento.

A função `end()` retorna um iterador para a posição imediatamente posterior ao último elemento.

```
1  vector<int> v = {10, 20, 30};
2
3  for (auto it = v.begin(); it != v.end(); ++it)
4      cout << *it << " ";
```

O operador `*` permite acessar o elemento indicado pelo iterador.

4.2 Iteradores reversos

Alguns containers oferecem iteradores para percorrimento em ordem inversa.

```
1  for (auto it = v.rbegin(); it != v.rend(); ++it)
2      cout << *it << " ";
```

Para:

```
1  vector<int> v = {10, 20, 30};
```

a saída será:

```
1  30 20 10
```

4.3 Categorias de iteradores

Os iteradores podem oferecer diferentes formas de movimentação.

Para programação competitiva, duas categorias são particularmente importantes:

- **bidirecionais**: permitem avançar e retroceder;
- **acesso aleatório**: permitem deslocamentos diretos entre posições.

Os iteradores de `vector` e `deque` permitem acesso aleatório. Os iteradores de `set` e `map` são bidirecionais.

Essa diferença é importante porque alguns algoritmos exigem iteradores de acesso aleatório. A função `sort`, por exemplo, pode ser utilizada diretamente com `vector`, mas não com `set`.

4.4 Iteradores retornados por operações

Muitas operações retornam iteradores.

```
auto it = s.find(x);

if (it != s.end())
    cout << "Encontrado";
```

De forma semelhante:

```
auto it = lower_bound(v.begin(), v.end(), x);
```

O uso de `auto` evita a necessidade de escrever explicitamente o tipo completo do iterador.

5 Adaptadores de containers

Adaptadores oferecem uma interface específica utilizando internamente outro container.

Os principais adaptadores utilizados em programação competitiva são:

- `stack`;
- `queue`;
- `priority_queue`.

5.1 `stack`

A `stack` implementa uma pilha seguindo a política LIFO (*Last In, First Out*).

Biblioteca

```
#include <stack>
```

Principais operações

Operação	Descrição	Complexidade
`push(x)`	empilha um elemento	$O(1)$
`pop()`	remove o topo	$O(1)$
`top()`	consulta o topo	$O(1)$
`size()`	quantidade de elementos	$O(1)$
`empty()`	verifica se está vazia	$O(1)$

Exemplo

```
#include <iostream>
#include <stack>

using namespace std;

int main() {
    stack<int> p;

    p.push(10);
    p.push(20);
    p.push(30);

```

```
    while (!p.empty()) {
        cout << p.top() << " ";
        p.pop();
    }

    return 0;
}
```

5.2 queue

A queue implementa uma fila seguindo a política FIFO (*First In, First Out*).

Biblioteca

```
#include <queue>
```

Principais operações

Operação	Descrição	Complexidade
`push(x)`	insere no final	$O(1)$
`pop()`	remove do início	$O(1)$
`front()`	primeiro elemento	$O(1)$
`back()`	último elemento	$O(1)$
`size()`	quantidade de elementos	$O(1)$
`empty()`	verifica se está vazia	$O(1)$

Exemplo

```
#include <iostream>
#include <queue>

using namespace std;

int main() {
    queue<int> f;

    f.push(10);
    f.push(20);
    f.push(30);

    while (!f.empty()) {
        cout << f.front() << " ";
        f.pop();
    }

    return 0;
}
```

5.3 priority_queue

A priority_queue mantém disponível o elemento de maior prioridade. Por padrão, o maior valor possui maior prioridade.

Biblioteca

```
#include <queue>
```

Principais operações

Operação	Descrição	Complexidade
`push(x)`	insere elemento	$O(\log n)$
`pop()`	remove maior prioridade	$O(\log n)$
`top()`	consulta maior prioridade	$O(1)$
`empty()`	verifica se está vazia	$O(1)$

Exemplo

```
#include <iostream>
#include <queue>

using namespace std;

int main() {
    priority_queue<int> fila;

    fila.push(20);
    fila.push(50);
    fila.push(10);
    fila.push(40);

    while (!fila.empty()) {
        cout << fila.top() << " ";
        fila.pop();
    }

    return 0;
}
```

Para fazer com que o menor elemento possua maior prioridade:

```
priority_queue<
    int,
    vector<int>,
    greater<int>
> fila;
```

6 Estruturas auxiliares

6.1 `pair`

A classe `pair` permite armazenar dois valores associados, possivelmente de tipos diferentes.

Biblioteca

```
#include <utility>
```

Declaração

```
pair<int, int> p;
pair<int, string> aluno;
```

Os elementos são acessados por:

```
p.first
p.second
```

Exemplo

```
#include <iostream>
#include <utility>

using namespace std;

int main() {
    pair<int, int> p = {10, 20};

    cout << p.first << " "
         << p.second << endl;

    return 0;
}
```

Em programação competitiva, `pair` aparece frequentemente em:

```
vector<pair<int, int>>
priority_queue<pair<int, int>>
map<int, pair<int, int>>
```

6.2 `tuple`

A classe `tuple` permite armazenar uma quantidade fixa de valores, possivelmente de tipos diferentes.

Biblioteca

```
#include <tuple>
```

Exemplo

```
#include <iostream>
#include <tuple>

using namespace std;

int main() {
    tuple<int, int, int> t = {10, 20, 30};

    cout << get<0>(t) << " ";
    cout << get<1>(t) << " ";
    cout << get<2>(t) << endl;

    return 0;
}
```

7 Objetos-função e comparadores

Algumas estruturas e algoritmos permitem alterar o critério utilizado para comparar seus elementos.

A biblioteca padrão fornece objetos-função como `less` e `greater`.

7.1 `less`

O `less` é um objeto-função que realiza uma comparação utilizando o operador `<`. Assim como `greater`, ele é parametrizado pelo tipo dos elementos comparados. De forma geral, seu uso pode ser representado por:

```
1 less<T>()
```

em que `T` é o tipo dos elementos comparados.

Por exemplo, para valores do tipo `int`, utiliza-se `less<int>()`:

```
1 sort(v.begin(), v.end(), less<int>());
```

Nesse caso, os elementos são ordenados em ordem crescente. Como esse já é o critério de comparação padrão utilizado por `sort`, o comando anterior é equivalente a:

```
1 sort(v.begin(), v.end());
```

Por essa razão, o uso explícito de `less` com `sort` não é normalmente necessário.

7.2 `greater`

O `greater` realiza a comparação utilizando o operador `>`, invertendo o critério apresentado anteriormente para `less`. De forma geral, seu uso pode ser representado por:

```
1 greater<T>()
```

Por exemplo, para ordenar um `vector<int>` em ordem decrescente:

```
1 vector<int> v = {10, 30, 20};
2
3 sort(v.begin(), v.end(), greater<int>());
```

Após a ordenação, o vetor conterá:

```
1 30 20 10
```

O `greater` também pode ser utilizado para definir o critério de uma fila de prioridade. Por padrão, uma `priority_queue` mantém o maior elemento no topo. Para manter o menor elemento no topo, pode-se utilizar:

```
1 priority_queue<
2     int,
3     vector<int>,
4     greater<int>
5 > fila;
```

Nesse caso, a fila armazena valores do tipo `int` e utiliza `greater<int>` como critério de comparação, fazendo com que o menor elemento fique disponível no topo da fila.

7.3 Comparadores personalizados

Quando a comparação desejada não corresponde à ordem natural dos elementos, pode-se definir uma função de comparação.

```
bool compara(pair<int, int> a,
             pair<int, int> b) {
    return a.second < b.second;
}
```

Ela pode ser utilizada com `sort`:

```
sort(v.begin(), v.end(), compara);
```

Por exemplo:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

bool compara(pair<int, int> a,
             pair<int, int> b) {
    return a.second < b.second;
}

int main() {
    vector<pair<int, int>> v = {
        {1, 30},
        {2, 10},
        {3, 20}
    };

    sort(v.begin(), v.end(), compara);

    for (auto p : v)
        cout << p.first << " "
             << p.second << endl;

    return 0;
}
```

8 Algoritmos

Grande parte dos algoritmos da STL está disponível no cabeçalho:

```
#include <algorithm>
```

Os algoritmos normalmente recebem intervalos delimitados por iteradores.

8.1 Algoritmos de busca e consulta

8.1.1 `find`

Procura um elemento em um intervalo.

```
auto it = find(v.begin(), v.end(), x);
if (it != v.end())
    cout << "Encontrado";
```

Complexidade:

$$O(n)$$

8.1.2 `count`

Conta quantas vezes determinado valor aparece.

```
int qtd = count(v.begin(), v.end(), x);
```

Complexidade:

$$O(n)$$

8.1.3 `min_element` e `max_element`

Retornam iteradores para o menor e o maior elemento.

```
auto menor = min_element(v.begin(), v.end());
auto maior = max_element(v.begin(), v.end());

cout << *menor << endl;
cout << *maior << endl;
```

Complexidade:

$$O(n)$$

8.2 Ordenação e busca binária

8.2.1 `sort`

Ordena os elementos de um intervalo.

```
sort(v.begin(), v.end());
```

Complexidade:

$$O(n \log n)$$

Para ordem decrescente:

```
sort(v.begin(), v.end(), greater<int>());
```

8.2.2 `binary_search`

Verifica se um valor está presente em um intervalo ordenado.

```
if (binary_search(v.begin(), v.end(), x))
    cout << "Encontrado";
```

Complexidade da busca em um `vector` ordenado:

$$O(\log n)$$

8.2.3 `lower_bound` e `upper_bound`

As funções `lower_bound` e `upper_bound` realizam buscas em uma sequência ordenada e retornam iteradores para posições que delimitam a ocorrência de um determinado valor.

Considere o vetor:

```
vector<int> v = {10, 20, 20, 20, 30, 40};
```

A função `lower_bound` retorna um iterador para o primeiro elemento **maior ou igual** a `x`:

```
auto it1 = lower_bound(v.begin(), v.end(), 20);
```

Nesse exemplo, `it1` aponta para o primeiro valor 20, localizado na posição 1.

A função `upper_bound`, por sua vez, retorna um iterador para o primeiro elemento **maior** que `x`:

```
auto it2 = upper_bound(v.begin(), v.end(), 20);
```

Nesse caso, `it2` aponta para o valor 30, localizado na posição 4.

Portanto, para o valor 20, temos:

Função	Elemento encontrado	Posição
`lower_bound`	primeiro elemento ≥ 20	1
`upper_bound`	primeiro elemento > 20	4

Como essas funções retornam iteradores, a posição correspondente em um `vector` pode ser obtida subtraindo-se `v.begin()`:

```
int p1 = it1 - v.begin();  // p1 = 1
int p2 = it2 - v.begin();  // p2 = 4
```

A diferença entre as duas posições também permite determinar quantas vezes `x` aparece em um vetor ordenado:

```
int qtd = it2 - it1;  // qtd = 3
```

Nesse exemplo, o valor 20 aparece três vezes.

As duas funções realizam a busca em $O(\log n)$ quando utilizadas sobre um `vector` ordenado.

8.3 Manipulação de sequências

8.3.1 `reverse`

Inverte a ordem dos elementos.

```
reverse(v.begin(), v.end());
```

Complexidade:

$$O(n)$$

8.3.2 `rotate`

Rotaciona os elementos de um intervalo.

```
rotate(v.begin(), v.begin() + k, v.end());
```

Por exemplo, para:

```
vector<int> v = {1, 2, 3, 4, 5};

rotate(v.begin(), v.begin() + 2, v.end());
```

o vetor passa a conter:

```
3 4 5 1 2
```

8.3.3 `unique`

A função `unique` atua sobre elementos iguais que aparecem **consecutivamente** em uma sequência. Ela mantém uma única ocorrência de cada grupo de elementos iguais e desloca os elementos resultantes para o início do intervalo.

Considere o vetor:

```
vector<int> v = {10, 10, 20, 20, 20, 30, 30, 40};

auto it = unique(v.begin(), v.end());
```

Após a execução de `unique`, o início do vetor conterá:

```
10 20 30 40 ...
```

Os quatro primeiros elementos correspondem ao resultado da operação. A função retorna um iterador para a posição imediatamente posterior a esse resultado:

```
auto it = unique(v.begin(), v.end());
//                          ^
//              novo final logico
```

É importante observar que `unique` não altera o tamanho do `vector`. Os elementos existentes entre `it` e `v.end()` continuam ocupando posições no vetor, mas seus valores não devem ser considerados como parte do resultado da operação.

Para remover efetivamente essas posições, pode-se utilizar `erase`:

```
auto it = unique(v.begin(), v.end());
v.erase(it, v.end());
```

Após essas operações, o vetor passa a conter:

```
10 20 30 40
```

Como `unique` considera apenas elementos iguais consecutivos, para eliminar todos os valores repetidos de um vetor que não está ordenado, uma estratégia comum é ordenar primeiro seus elementos:

```
vector<int> v = {30, 10, 20, 10, 30, 20, 40};

sort(v.begin(), v.end());

auto it = unique(v.begin(), v.end());
v.erase(it, v.end());
```

Ao final, o vetor conterá apenas valores distintos:

```
10 20 30 40
```

8.4 Permutações

Os algoritmos `next_permutation` e `prev_permutation` permitem percorrer as diferentes permutações dos elementos de uma sequência.

8.4.1 `next_permutation`

A função `next_permutation` reorganiza os elementos para formar a próxima permutação em ordem lexicográfica.

Por exemplo, considere:

```
vector<int> v = {1, 2, 3};

next_permutation(v.begin(), v.end());
```

Após a execução, o vetor conterá:

```
1 3 2
```

Uma nova chamada a `next_permutation` produzirá:

```
2 1 3
```

Para gerar todas as permutações de uma sequência, seus elementos devem estar inicialmente em ordem crescente:

```
vector<int> v = {1, 2, 3};

sort(v.begin(), v.end());

do {
    for (int x : v)
        cout << x << " ";

    cout << endl;

} while (next_permutation(v.begin(), v.end()));
```

A saída será:

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

Para uma sequência com n elementos distintos, existem $n!$ permutações. Portanto, gerar todas as permutações é viável apenas para valores relativamente pequenos de n.

8.4.2 `prev_permutation`

A função `prev_permutation` realiza a operação inversa, formando a permutação imediatamente anterior em ordem lexicográfica.

Por exemplo:

```
vector<int> v = {3, 1, 2};

prev_permutation(v.begin(), v.end());
```

Após a execução, o vetor conterá:

```
2 3 1
```

Assim, enquanto `next_permutation` percorre as permutações no sentido crescente da ordem lexicográfica, `prev_permutation` permite percorrê-las no sentido contrário.

8.5 Operações numéricas

Algumas operações numéricas estão disponíveis em:

```
#include <numeric>
```

8.5.1 `accumulate`

Acumula os valores de um intervalo.

```
int soma =
    accumulate(v.begin(), v.end(), 0);
```

Complexidade:

$$O(n)$$

Para valores que podem produzir uma soma grande, pode-se utilizar:

```
long long soma =
    accumulate(v.begin(), v.end(), 0LL);
```

O sufixo `LL` indica que o literal inteiro deve ser tratado como um valor do tipo `long long`. Assim, `0LL` representa o valor zero do tipo `long long`.

Esse detalhe é importante porque o terceiro argumento de `accumulate` determina o tipo utilizado para realizar a acumulação. Portanto, mesmo que o resultado seja armazenado em uma variável `long long`, utilizar simplesmente `0` faria com que a soma fosse realizada utilizando o tipo `int`.

Por isso, quando a soma puder ultrapassar os limites de `int`, deve-se utilizar `0LL` como valor inicial.

8.5.2 `iota`

Preenche um intervalo com valores consecutivos.

```
vector<int> v(5);

iota(v.begin(), v.end(), 1);
```

O vetor passa a conter:

```
1 2 3 4 5
```

8.6 Operações sobre conjuntos

A STL também fornece algoritmos para operações sobre sequências ordenadas que representam conjuntos.

8.6.1 `set_union`

Calcula a união de dois intervalos ordenados.

```
set_union(
    a.begin(), a.end(),
    b.begin(), b.end(),
    back_inserter(c)
);
```

8.6.2 `set_intersection`

Calcula a interseção.

```
set_intersection(
    a.begin(), a.end(),
    b.begin(), b.end(),
    back_inserter(c)
);
```

8.6.3 `set_difference`

Calcula os elementos presentes no primeiro intervalo e ausentes no segundo.

```
set_difference(
    a.begin(), a.end(),
    b.begin(), b.end(),
    back_inserter(c)
);
```

Esses algoritmos exigem que os intervalos de entrada estejam ordenados.

Exemplo

O programa a seguir aplica as três operações sobre os mesmos conjuntos. Como os algoritmos trabalham com intervalos ordenados, os elementos dos vetores `a` e `b` são apresentados em ordem crescente.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>

using namespace std;

int main() {
    vector<int> a = {1, 2, 3, 5};
    vector<int> b = {2, 3, 4, 6};

    vector<int> uniao;
    vector<int> intersecao;
    vector<int> diferenca;

    set_union(
        a.begin(), a.end(),
        b.begin(), b.end(),
        back_inserter(uniao)
    );

    set_intersection(
        a.begin(), a.end(),
        b.begin(), b.end(),
        back_inserter(intersecao)
    );

    set_difference(
        a.begin(), a.end(),
        b.begin(), b.end(),
        back_inserter(diferenca)
    );

    cout << "Uniao: ";
    for (int x : uniao)
        cout << x << " ";
    cout << endl;

    cout << "Intersecao: ";
    for (int x : intersecao)
        cout << x << " ";
    cout << endl;
```

```
43
44      cout << "Diferenca A - B: ";
45      for (int x : diferenca)
46          cout << x << " ";
47      cout << endl;
48
49      return 0;
50  }
```

A saída será:

```
1  Uniao: 1 2 3 4 5 6
2  Intersecao: 2 3
3  Diferenca A - B: 1 5
```

A função `back_inserter` cria um iterador que insere cada elemento produzido pelo algoritmo no final do vetor de destino, utilizando `push_back`. Dessa forma, não é necessário definir previamente o tamanho dos vetores que receberão os resultados.

Observe também que `set_difference` não é uma operação simétrica. No exemplo, ela calcula $A - B$, isto é, os elementos que pertencem a `a` e não pertencem a `b`. Para calcular $B - A$, basta inverter a ordem dos dois intervalos na chamada da função.

9 Escolhendo estruturas e algoritmos

Uma das habilidades importantes em programação competitiva é reconhecer quais estruturas e algoritmos são adequados às operações exigidas pelo problema.

Necessidade	Recurso sugerido
Sequência com acesso por índice	`vector`
Inserção e remoção nas duas extremidades	`deque`
Pilha	`stack`
Fila	`queue`
Acessar repetidamente maior ou menor elemento	`priority_queue`
Valores distintos e ordenados	`set`
Valores ordenados com repetição	`multiset`
Associação chave–valor ordenada	`map`
Busca esperada em tempo constante	`unordered_set` ou `unordered_map`
Ordenar uma sequência	`sort`
Buscar em sequência ordenada	`binary_search`
Encontrar posição em sequência ordenada	`lower_bound` e `upper_bound`
Eliminar repetições de um vetor	`sort` + `unique`
Gerar permutações	`next_permutation`

10 Problemas de programação competitiva

Nesta seção são apresentados exemplos de problemas que ilustram a utilização dos recursos da STL em situações comuns de programação competitiva.

O objetivo é mostrar não apenas a sintaxe das estruturas, mas também como identificar situações em que seu uso simplifica a solução.

10.1 Ordenando as pontuações

Durante uma competição, N equipes receberam diferentes pontuações. Dadas as pontuações obtidas, apresente os valores em ordem decrescente.

Entrada

A primeira linha contém um inteiro N. A segunda linha contém N inteiros representando as pontuações.

Exemplo de entrada

```
5
120 350 210 180 300
```

Exemplo de saída

```
350 300 210 180 120
```

Solução

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>

using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> pontos(n);

    for (int i = 0; i < n; i++)
        cin >> pontos[i];

    sort(
        pontos.begin(),
        pontos.end(),
        greater<int>()
    );

    for (int x : pontos)
        cout << x << " ";

    cout << endl;

    return 0;
}
```

A solução utiliza `vector` para armazenamento e `sort` para ordenação. Sua complexidade é $O(n \log n)$.

10.2 Contando valores distintos

Dada uma sequência com N números inteiros, determine quantos valores distintos aparecem.

Exemplo de entrada

```
8
10 20 10 30 20 40 30 50
```

Exemplo de saída

```
5
```

Solução

```
#include <iostream>
#include <set>

using namespace std;

int main() {
    int n, x;
    cin >> n;

    set<int> valores;

    for (int i = 0; i < n; i++) {
        cin >> x;
        valores.insert(x);
    }

    cout << valores.size() << endl;

    return 0;
}
```

O `set` elimina naturalmente as repetições. Como cada inserção possui complexidade $O(\log n)$, a solução possui complexidade $O(n \log n)$.

10.3 Frequência dos valores

Dada uma sequência de N números inteiros, determine quantas vezes cada valor aparece. Os valores devem ser apresentados em ordem crescente.

Exemplo de entrada

```
8
3 1 2 3 2 3 1 5
```

Exemplo de saída

```
1 2
2 2
3 3
5 1
```

Solução

```
#include <iostream>
#include <map>

using namespace std;

int main() {
    int n, x;
    cin >> n;

```

```
    map<int, int> freq;

    for (int i = 0; i < n; i++) {
        cin >> x;
        freq[x]++;
    }

    for (auto p : freq)
        cout << p.first << " "
             << p.second << endl;

    return 0;
}
```

O `map` associa cada valor à sua frequência e mantém as chaves ordenadas.

10.4 Atendimento por prioridade

Cada tarefa possui uma prioridade inteira. Quanto maior o valor, maior a prioridade. Determine a ordem em que as tarefas serão processadas.

Exemplo de entrada

```
5
20 50 10 40 30
```

Exemplo de saída

```
50 40 30 20 10
```

Solução

```
#include <iostream>
#include <queue>

using namespace std;

int main() {
    int n, x;
    cin >> n;

    priority_queue<int> fila;

    for (int i = 0; i < n; i++) {
        cin >> x;
        fila.push(x);
    }

    while (!fila.empty()) {
        cout << fila.top() << " ";
        fila.pop();
    }

    cout << endl;

    return 0;
}
```

10.5 Primeira posição possível

Uma sequência de N números está ordenada em ordem crescente. Determine a primeira posição em que um valor x poderia ser inserido sem alterar a ordenação.

Considere as posições numeradas a partir de zero.

Exemplo de entrada

```
6
10 20 20 30 40 50
25
```

Exemplo de saída

```
3
```

Solução

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> v(n);

    for (int i = 0; i < n; i++)
        cin >> v[i];

    int x;
    cin >> x;

    int pos =
        lower_bound(v.begin(), v.end(), x)
        - v.begin();

    cout << pos << endl;

    return 0;
}
```

Como o vetor já está ordenado, `lower_bound` realiza a busca em $O(\log n)$.

10.6 Busca em largura em um grafo

Um grafo pode ser representado por uma lista de adjacências utilizando `vector`. Uma `queue` pode então controlar a ordem dos vértices processados durante uma busca em largura.

```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

int main() {
```

```
    int n = 6;

    vector<vector<int>> G(n);

    G[0] = {1, 2};
    G[1] = {0, 3, 4};
    G[2] = {0, 5};
    G[3] = {1};
    G[4] = {1};
    G[5] = {2};

    vector<bool> visitado(n, false);
    queue<int> fila;

    visitado[0] = true;
    fila.push(0);

    while (!fila.empty()) {
        int v = fila.front();
        fila.pop();

        cout << v << " ";

        for (int u : G[v]) {
            if (!visitado[u]) {
                visitado[u] = true;
                fila.push(u);
            }
        }
    }

    cout << endl;

    return 0;
}
```

Para um grafo com V vértices e E arestas, a busca possui complexidade $O(V + E)$ quando o grafo é representado por listas de adjacências.

10.7 Menor caminho com fila de prioridade

O algoritmo de Dijkstra para caminhos mínimos em grafos com pesos não negativos pode utilizar uma `priority_queue` para selecionar eficientemente o próximo vértice.

```
#include <iostream>
#include <vector>
#include <queue>
#include <utility>
#include <functional>
#include <climits>

using namespace std;

int main() {
    int n = 5;

    vector<vector<pair<int, int>>> G(n);

```

```
    G[0].push_back({1, 4});
    G[0].push_back({2, 1});
    G[2].push_back({1, 2});
    G[1].push_back({3, 1});
    G[2].push_back({3, 5});
    G[3].push_back({4, 3});

    vector<int> dist(n, INT_MAX);

    priority_queue<
        pair<int, int>,
        vector<pair<int, int>>,
        greater<pair<int, int>>
    > fila;

    dist[0] = 0;
    fila.push({0, 0});

    while (!fila.empty()) {
        int d = fila.top().first;
        int v = fila.top().second;
        fila.pop();

        if (d != dist[v])
            continue;

        for (auto aresta : G[v]) {
            int u = aresta.first;
            int peso = aresta.second;

            if (dist[v] + peso < dist[u]) {
                dist[u] = dist[v] + peso;
                fila.push({dist[u], u});
            }
        }
    }

    for (int v = 0; v < n; v++)
        cout << v << ": "
             << dist[v] << endl;

    return 0;
}
```

Este exemplo combina diferentes componentes da STL: `vector`, `pair` e `priority_queue`.

10.8 Soma de prefixos

Um *vector* pode ser utilizado para armazenar estruturas auxiliares que tornam mais eficientes determinadas operações sobre sequências de valores. Um exemplo é o vetor de soma de prefixos, utilizado para responder consultas de soma em intervalos.

Considere o vetor:

$$v = [1, 3, 4, 8, 6, 1, 4, 2].$$

Pode-se construir um segundo vetor em que cada posição armazena a soma dos elementos desde o início do vetor:

$$prefixo = [1, 4, 8, 16, 22, 23, 27, 29].$$

A implementação utilizando `vector` é:

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> v = {1, 3, 4, 8, 6, 1, 4, 2};
    vector<int> prefixo(v.size());

    prefixo[0] = v[0];

    for (int i = 1; i < v.size(); i++)
        prefixo[i] = prefixo[i - 1] + v[i];

    int a = 3;
    int b = 6;

    int soma = prefixo[b];

    if (a > 0)
        soma -= prefixo[a - 1];

    cout << soma << endl;

    return 0;
}
```

A construção do vetor de prefixos é realizada em $O(n)$. Depois disso, a soma de qualquer intervalo $[a, b]$ pode ser obtida em $O(1)$:

$$\text{soma}(a, b) = \text{prefixo}[b] - \text{prefixo}[a - 1].$$

Quando $a = 0$, basta utilizar `prefixo[b]`.

11 Referência rápida

Esta seção reúne algumas das operações mais utilizadas para consulta rápida durante a resolução de problemas.

`vector`

```cpp
vector<int> v;

v.push_back(x);
v.pop_back();

v[i];
v.front();
v.back();

v.size();
v.empty();
v.clear();
```

stack

```
stack<int> p;

p.push(x);
p.pop();
p.top();

p.size();
p.empty();
```

queue

```
queue<int> f;

f.push(x);
f.pop();

f.front();
f.back();

f.size();
f.empty();
```

priority_queue

```
priority_queue<int> p;

p.push(x);
p.pop();
p.top();

p.size();
p.empty();
```

set

```
set<int> s;

s.insert(x);
s.erase(x);
s.find(x);
s.count(x);

s.size();
s.empty();
s.clear();
```

map

```
map<int, int> m;

m[chave] = valor;
```

```
4
5  m.find(chave);
6  m.count(chave);
7  m.erase(chave);
8
9  m.size();
10 m.empty();
11 m.clear();
```

Algoritmos

```
1  sort(v.begin(), v.end());
2
3  reverse(v.begin(), v.end());
4
5  find(v.begin(), v.end(), x);
6
7  count(v.begin(), v.end(), x);
8
9  min_element(v.begin(), v.end());
10
11 max_element(v.begin(), v.end());
12
13 binary_search(v.begin(), v.end(), x);
14
15 lower_bound(v.begin(), v.end(), x);
16
17 upper_bound(v.begin(), v.end(), x);
18
19 next_permutation(v.begin(), v.end());
20
21 accumulate(v.begin(), v.end(), 0);
```

12 Considerações finais

A STL oferece estruturas de dados e algoritmos que permitem implementar soluções de forma concisa e eficiente. Em programação competitiva, sua utilização reduz a necessidade de implementar repetidamente estruturas básicas e permite concentrar maior atenção no algoritmo necessário para resolver o problema.

O uso eficiente da STL não depende da memorização de todas as operações disponíveis. É mais importante compreender as características dos principais containers, as operações que oferecem e suas complexidades.

Também é importante perceber que os componentes da STL podem ser combinados. Um `vector` pode armazenar elementos que serão ordenados por `sort`; um `map` pode associar valores às suas frequências; e estruturas como `vector`, `pair` e `priority_queue` podem ser utilizadas conjuntamente na implementação de algoritmos de grafos.

Com a prática, a escolha dessas estruturas e algoritmos passa a fazer parte naturalmente do processo de elaboração de uma solução para problemas de programação competitiva.